pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Clean and Dirty Price

WEEK 2 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: the quote and the cash

The Desk Lead's Note

We buy 1,000,000 of Quorvane Energy at its issue yield, 5.40 percent, settling Monday 30 November. Tell me the quote and the cash that leaves the account. Then give "bondmath" a price for any settlement date and a yield from any quote, both tied to Excel. Last, September's blotter shows principal only: put the accrued back on all 374 trades and tell me what the cash really was.

The issuers are invented; every statement describes the data.

cusip	issuer	coupon_pct	maturity	yield_pct
99001BZA7	Quorvane Energy	5.250	2031-09-30	5.400
99002CZA4	Dunmarrow Rail	6.125	2033-09-30	6.125
99003DZA1	Pellwick Paper	4.750	2029-09-30	4.600
99004EZA8	Halvering Foods	7.500	2036-09-30	7.700

```
306 reference rows, 8 of them in week2
```

```
..................   [100%]
18 passed in 0.50s
```

The module starts the day as notebook 07 left it, with 18 tests passing.

Notebook 02 priced on a coupon date; today the next coupon is four months away.

8.1 The quote and the invoice

The market quotes the clean price; the buyer pays the dirty price, clean plus accrued (notebook 02's terms). The invoice is the cash that settles the trade: the dirty price applied to the face.

What Notebooks 02 and 07 Already Know

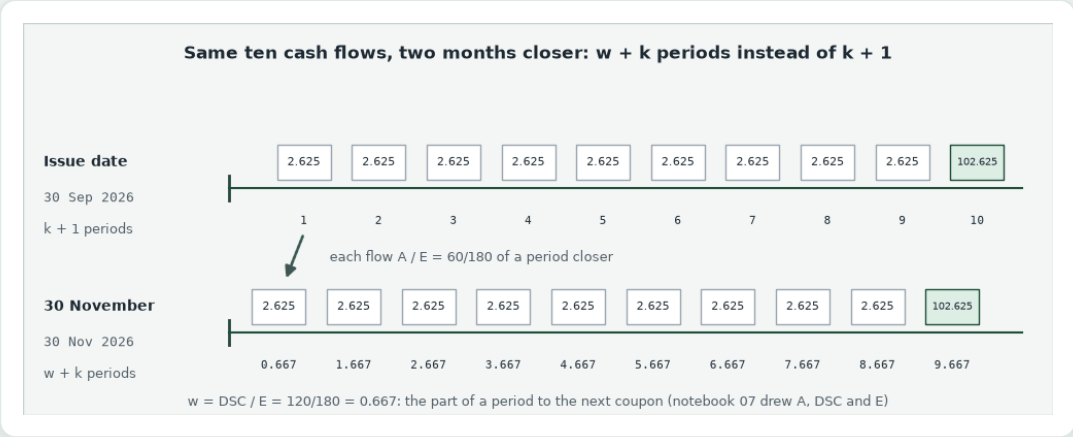
```
issue_price = bondmath.price_from_yield(5.25, 5.40, 5)
quorvane_accrued = bondmath.accrued_interest(settlement, quorvane_maturity, 5.25)
print(f'Price on 30 September: {issue_price:.6f}')
print(f'Accrued on 30 November: {quorvane_accrued}')
Price on 30 September: 99.350327
Accrued on 30 November: 0.875
```

The ticket shows the clean price; settlement moves the dirty amount.

Excel side by side: the quote, and the invoice price

```
The quote
=PRICE(DATE(2026,11,30),DATE(2031,9,30),5.25/100,5.4/100,100,2,0)
The invoice price
=PRICE(DATE(2026,11,30),DATE(2031,9,30),5.25/100,5.4/100,100,2,0)+100*5.25/100/2*COUPDAYBS(DATE(2026,11,30),
DATE(2031,9,30),2,0)/COUPDAYS(DATE(2026,11,30),DATE(2031,9,30),2,0)
```

8.2 Discounting from settlement, w = DSC / E



Notebook 07 split Quorvane's period into A, DSC and E. The next coupon is now only w = DSC / E of a period away, and cash flow k is w + k periods away.

```
days_accrued, days_in_period, days_to_next = bondmath.coupon_period_days(settlement,
                                                                         quorvane_maturity)
w = days_to_next / days_in_period
print(f'A = {days_accrued}, E = {days_in_period}, DSC = {days_to_next}, w = {w:.6f}')
A = 60, E = 180, DSC = 120, w = 0.666667
```

The Ten Cash Flows, Added Up

```
dates = bondmath.coupon_dates(settlement, quorvane_maturity)
coupons_left = len(dates) - 1
period_yield = 5.40 / 100 / 2
rows = []
for k in range(coupons_left):
    cash_flow = 5.25 / 2 + (100 if k == coupons_left - 1 else 0)
    rows.append({'coupon_date': dates[k + 1], 'periods': w + k, 'cash_flow': cash_flow,
                 'present_value': cash_flow / (1 + period_yield) ** (w + k)})
flows = pd.DataFrame(rows)
dirty_by_hand = flows['present_value'].sum()
clean_by_hand = dirty_by_hand - quorvane_accrued
assert abs(clean_by_hand - excel_clean) < 1e-9
```

The sum is the dirty price; take off the accrued and Excel's "PRICE" comes back to the tenth decimal. The market discounts first and subtracts accrued second.

The dirty price is the cash flows discounted from settlement; the quote is that less accrued.

The price written out in a sheet

Excel side by side. The price written out, in a blank sheet with k = 0 to 9 typed in A6:A15.

Job	Python	Excel
A, E, coupons left in B1:B3	`coupon_period_days, coupon_dates`	=COUPDAYBS(DATE(2026,11,30),DATE(2031,9,30),2,0), =COUPDAYS(…), =COUPNUM(…), the same arguments
w in B4	`w`	=(B2-B1)/B2
Each flow, B6 filled to B15	`flows['present_value']`	=IF(A6=B3-1,5.25/2+100,5.25/2)/(1+5.4/100/2)^(B4+A6)
Dirty price in B16	`dirty_by_hand`	=SUM(B6:B15)
Clean price in B17	`clean_by_hand`	=B16-100*5.25/100/2*B1/B2

```
Dirty price, one cell
=-PV(5.4/100/2,COUPNUM(DATE(2026,11,30),DATE(2031,9,30),2,0),5.25/2,100)*(1+5.4/100/2)^(COUPDAYBS(DATE(2026,
11,30),DATE(2031,9,30),2,0)/COUPDAYS(DATE(2026,11,30),DATE(2031,9,30),2,0))
```

Two Months Closer

Why did the quote rise? Grow notebook 02's price by the A / E of a period that every flow moved closer.

```
shifted = issue_price * (1 + period_yield) ** (days_accrued / days_in_period)
```

8.3 price and dirty_price, docstring first

```
%%writefile -a bondmath.py
def price(settlement: date, maturity: date, coupon_pct: float, yield_pct: float, frequency: int = 2,
          basis: str = "30/360") -> float:
    dates = coupon_dates(settlement, maturity, frequency)
    days_accrued, days_in_period, days_to_next = coupon_period_days(settlement, maturity, frequency, basis)
    coupons_left = len(dates) - 1
    w = days_to_next / days_in_period
    period_yield = yield_pct / 100 / frequency
    coupon = coupon_pct / frequency
    if coupons_left == 1:
        dirty = (100 + coupon) / (1 + w * period_yield)
    else:
        dirty = 0.0
        for k in range(coupons_left):
            cash_flow = coupon + 100 if k == coupons_left - 1 else coupon
            dirty += cash_flow / (1 + period_yield) ** (w + k)
    return dirty - accrued_interest(settlement, maturity, coupon_pct, frequency, basis)
def dirty_price(settlement: date, maturity: date, coupon_pct: float, yield_pct: float, frequency: int = 2,
                basis: str = "30/360") -> float:
    return (price(settlement, maturity, coupon_pct, yield_pct, frequency, basis)
            + accrued_interest(settlement, maturity, coupon_pct, frequency, basis))
```

"price" takes PRICE's arguments in Excel's order, without the redemption of 100, and returns the clean price. Its docstring states the units (percent in, per 100 out), the convention (w + k, the final period, clean = dirty - accrued) and the Excel formula with its "/100". The loop is 8.2; the "if coupons_left == 1" branch is 8.5. "dirty_price" adds "accrued_interest" back, so the two can never disagree about the accrued.

Two tests from the reference file

"bond_inputs(row)" (notebook 07) returns a tuple of six arguments. A "*" before it in a call hands them over one by one: "price(*bond_inputs(row))" is "price(settlement, maturity, coupon_pct, yield_pct, frequency, basis)".

```
%%writefile -a test_bondmath.py
def test_price_matches_excel():
    for row in REFERENCE:
        result = bondmath.price(*bond_inputs(row))
        assert result == pytest.approx(float(row["price"]), abs=1e-6), row["case_id"]
def test_dirty_is_clean_plus_accrued():
    for row in REFERENCE:
        result = bondmath.dirty_price(*bond_inputs(row))
        assert result == pytest.approx(float(row["price"]) + float(row["accrued"]), abs=1e-6), row["case_id"]
```

```
....................   [100%]
20 passed in 0.50s
```

"price" matches Excel's "PRICE", and "dirty_price" "PRICE" plus accrued, on all 306 reference rows.

issuer	clean	excel	dirty
Pellwick Paper	100.388247	100.388247	101.179914
Halvering Foods	98.622491	98.622491	99.872491

All four clean prices are Excel's "PRICE".

8.4 On a coupon date it equals notebook 02

```
days = [date(2026, 9, 30) + timedelta(days=i) for i in range(366)]
clean_prices = [bondmath.price(d, quorvane_maturity, 5.25, 5.40) for d in days]
dirty_prices = [bondmath.dirty_price(d, quorvane_maturity, 5.25, 5.40) for d in days]
print(f'Dirty on 30 and 31 March 2027: {dirty_prices[181]:.6f}, {dirty_prices[182]:.6f}')
Dirty on 30 and 31 March 2027: 102.032786, 99.407786
```

The dirty price falls by exactly 2.625 on 31 March: under 30/360, 30 March already counts all 180 days. The clean price moves smoothly towards 100, so a change in the quote is a change in value, not in the calendar. That is why the market quotes clean.

The Issue Date

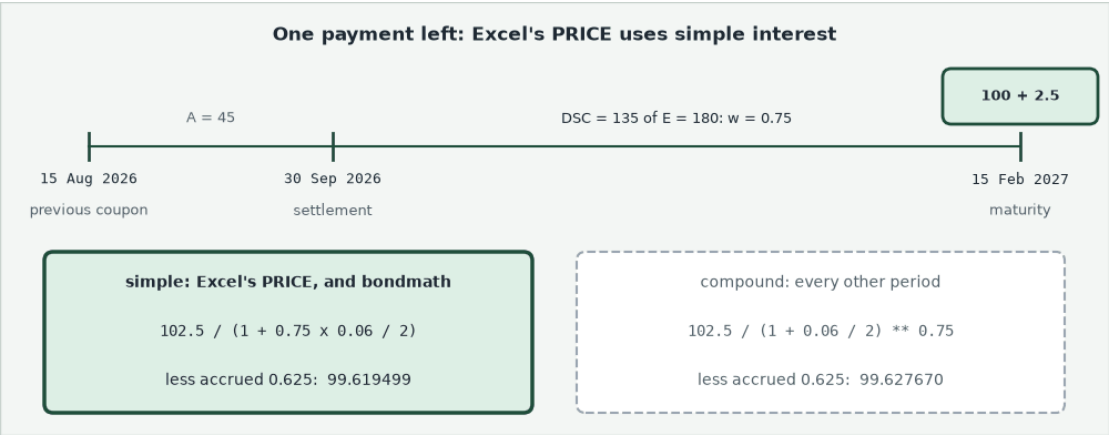
```
print(bondmath.coupon_period_days(date(2026, 9, 30), quorvane_maturity))
on_issue = [bondmath.price(date(2026, 9, 30), m, c, y)
            for m, c, y in zip(maturities, coupons, yields)]
from_02 = [bondmath.price_from_yield(c, y, m.year - 2026)
           for m, c, y in zip(maturities, coupons, yields)]
print(max([abs(a - b) for a, b in zip(on_issue, from_02)]))
(0, 180, 180)
0.0
```

A = 0 and DSC = E, so w = 1 and nothing is taken off: the same four prices. A new function must keep the old answers.

```
%%writefile -a test_bondmath.py
def test_price_on_coupon_date_equals_week1():
    coupon_date_rows = [row for row in REFERENCE if row["coupdaybs"] == "0"]
    assert coupon_date_rows, "no coupon-date rows in the reference file"
    for row in coupon_date_rows:
        settlement, maturity, coupon_pct, yield_pct, frequency, basis = bond_inputs(row)
        years = int(row["coupnum"]) / frequency
        week1_price = bondmath.price_from_yield(coupon_pct, yield_pct, years, frequency)
        assert bondmath.price(settlement, maturity, coupon_pct, yield_pct, frequency, basis) == pytest.approx(
            week1_price, abs=1e-9), row["case_id"]
```

The new function keeps every old answer, and the quote does not jump on coupon day.

8.5 The final coupon period: simple interest



With one coupon left, Excel's "PRICE" discounts the last payment with simple interest: dirty = (100 + coupon / f) / (1 + w x y / f). None of the four bonds is there yet, so take the reference row "edge_final_period".

```
w = 0.75; Excel's PRICE 99.6194987775
Simple:   99.6194987775
Compound: 99.6276697643
```

"price" is Excel's number to the last digit; compounding would add 0.008171 per 100, 81.71 dollars on 1,000,000.

Excel side by side

```
Simple, written out
=(100+5/2)/(1+(COUPDAYS(DATE(2026,9,30),DATE(2027,2,15),2,0)-COUPDAYBS(DATE(2026,9,30),DATE(2027,2,15),2,
0))/COUPDAYS(DATE(2026,9,30),DATE(2027,2,15),2,0)*6/100/2)-100*5/100/2*COUPDAYBS(DATE(2026,9,30),DATE(2027,2,
15),2,0)/COUPDAYS(DATE(2026,9,30),DATE(2027,2,15),2,0)
Compound, written out
the same with (1+6/100/2)^(...) in place of (1+(...)*6/100/2)
```

Job	Python	Result
Simple, written out	`simple`	99.61949877750611, as PRICE
Compound, written out	`compound`	99.62766976428897

8.6 Undo a commit, keep the history

A colleague wants the final period "discounted like every other period", edits one line of "price", and commits without running the tests.

```
@@ -296,7 +296,7 @@ def price(settlement: date, maturity: date, coupon_pct: float, yield_pct: float,
-    dirty = (100 + coupon) / (1 + w * period_yield)
+    dirty = (100 + coupon) / (1 + period_yield) ** w
```

```
!git -C "{work}" commit -q -am "Discount the final period like every other period"
!git -C "{work}" log --oneline -3
04c5b5e Discount the final period like every other period
```

"git restore" (notebook 04) cannot help: the broken file is committed. "git revert" makes a new commit that does the opposite of an old one.

```
!git -C "{work}" revert --no-edit HEAD
!git -C "{work}" log --oneline -4
a23e982 Revert "Discount the final period like every other period"
04c5b5e Discount the final period like every other period
4ed07ee Add price and dirty_price, tested against Excel PRICE
98124d3 Notebook 07: days_30_360, coupon_period_days, and accrued_interest
```




```
.....................   [100%]
22 passed in 0.50s
```

"restore" undoes what is not committed; "revert" undoes what is and keeps the record. Never rewrite a shared history; add to it.

8.7 yield_to_maturity, as Excel's YIELD

```
%%writefile -a bondmath.py
def yield_to_maturity(settlement: date, maturity: date, coupon_pct: float, clean_price: float,
                      frequency: int = 2, basis: str = "30/360") -> float:
    if len(coupon_dates(settlement, maturity, frequency)) - 1 == 1:
        days_accrued, days_in_period, _ = coupon_period_days(settlement, maturity, frequency, basis)
        if basis == "30/360":
            days_to_maturity = days_30_360(settlement, maturity)
        else:
            days_to_maturity = (maturity - settlement).days
        coupon = coupon_pct / frequency
        paid = clean_price + coupon * days_accrued / days_in_period
        received = 100 + coupon
        return (received - paid) / paid * frequency * days_in_period / days_to_maturity * 100
    low_pct, high_pct = -5.0, 100.0
    highest_price = price(settlement, maturity, coupon_pct, low_pct, frequency, basis)
    lowest_price = price(settlement, maturity, coupon_pct, high_pct, frequency, basis)
    if not lowest_price <= clean_price <= highest_price:
        raise ValueError(f"price {clean_price} is outside {lowest_price:.6f} to {highest_price:.6f}, "
                         f"the prices at yields of {high_pct} and {low_pct} percent")
    while high_pct - low_pct > 1e-10:
        mid_pct = (low_pct + high_pct) / 2
        if price(settlement, maturity, coupon_pct, mid_pct, frequency, basis) > clean_price:
            low_pct = mid_pct
        else:
            high_pct = mid_pct
    return (low_pct + high_pct) / 2
```

From a clean price back to the yield, as Excel's "YIELD". The docstring states the units (percent a year), the bracket (-5 to 100 percent), the final-period formula, and YIELD's "*100". The search is notebook 04's bisection with "price" inside: ten clear lines read better than a general solver.

The Issue Yields Come Back

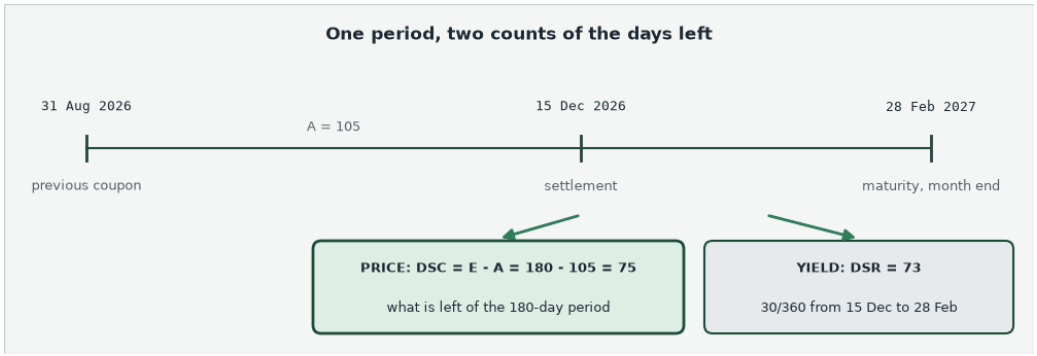
```
Largest gap: 1.9e-11 percent
```

When YIELD does not undo PRICE

In the final period "YIELD" uses a formula, with DSR, the days from settlement to maturity, counted on 30/360. "PRICE" uses DSC = E - A, and near a February month end the two differ.

```
PRICE at 5 percent: 100.289734
YIELD of that: 5.136986; ours: 5.136986
Priced again at that yield: 100.260880
```

```
E - A for PRICE: 75; DSR for YIELD: 73
```



Each function does what it documents, and "bondmath" follows Excel in both; no bond in the holdings file is in its final period.

Excel side by side

Job	Python	Excel	Result
DSR	`bondmath.days_30_360(...)`	=YEARFRAC(DATE(2026,12,15),DATE(2027,2,28),0)*360	73
E - A	`e - a`	=COUPDAYS(DATE(2026,12,15),DATE(2027,2,28),2,0)-COUPDAYBS(DATE(2026,12,15),DATE(2027,2,28),2,0)	75

```
......................   [100%]
23 passed in 0.50s
```

From any quote to its yield, as Excel's "YIELD", final period included.

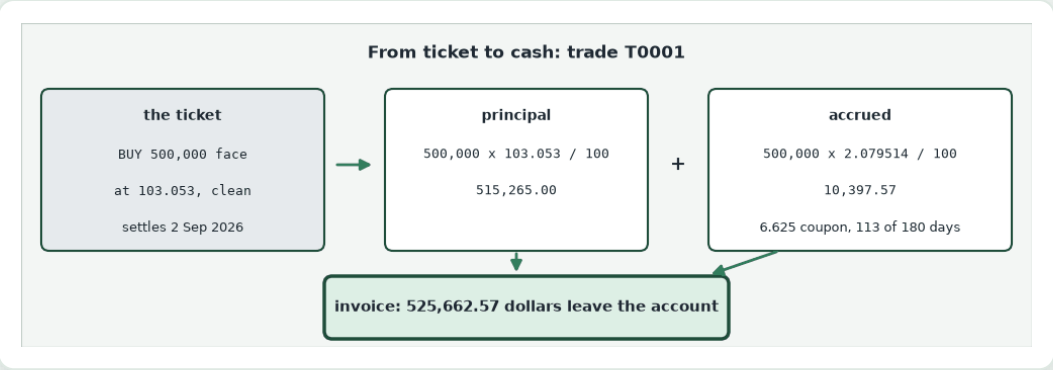
8.8 The September blotter's invoices

Course 1's Trade Blotter case study read this file. Its "price" is the clean price and its "principal" is face times that price: no accrued.

```
374 trades, settling 2026-09-02 to 2026-10-01
```

```
settle_dates = pd.to_datetime(trades['settle_date']).dt.date
maturity_dates = pd.to_datetime(trades['maturity']).dt.date
trades['accrued_per_100'] = [bondmath.accrued_interest(s, m, c)
                             for s, m, c in zip(settle_dates, maturity_dates, trades['coupon'])]
trades['accrued_dollars'] = trades['quantity'] * trades['accrued_per_100'] / 100
trades['invoice_dollars'] = trades['principal'] + trades['accrued_dollars']
```

trade_id	quantity	principal	accrued_dollars	invoice_dollars
T0001	500000	515265.00	10397.569444	525662.569444
T0002	245000	233813.30	2825.156250	236638.456250
T0003	25000	24468.75	471.701389	24940.451389



T0001 settles for 525,662.57 dollars: 515,265.00 of principal and 10,397.57 of accrued. The two trades that accrued nothing settled on a coupon date.

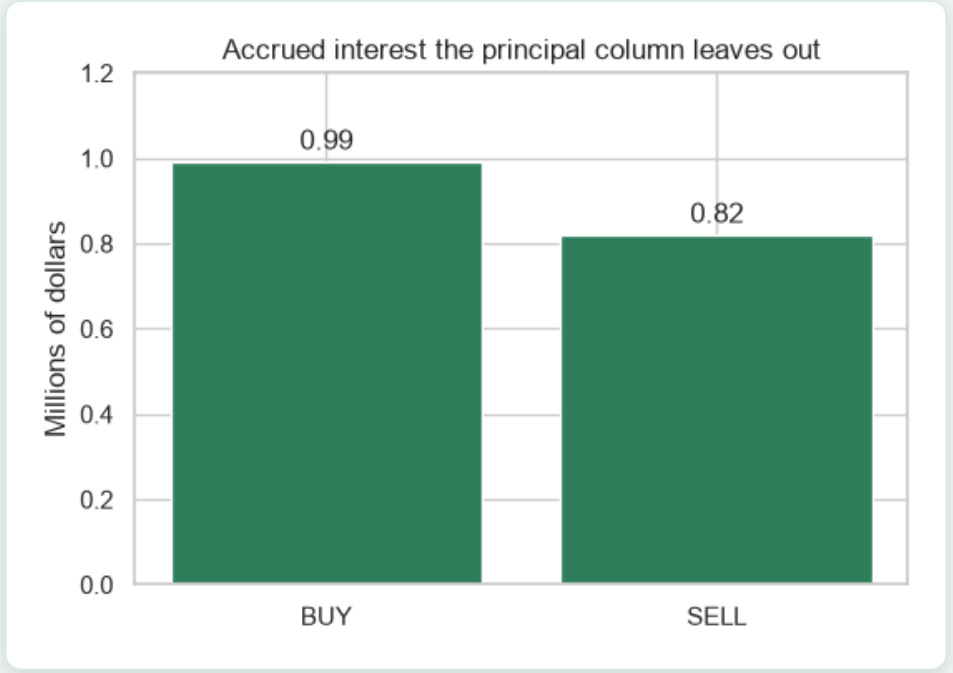
What the month's trades really settled for

side	principal	accrued_dollars	invoice_dollars
BUY	56,556,005.45	990,620.95	57,546,626.40
SELL	60,917,743.90	819,168.26	61,736,912.16
All	117,473,749.35	1,809,789.22	119,283,538.57

Excel side by side

```
T0001's invoice
=500000*(103.053+100*6.625/100/2*COUPDAYBS(DATE(2026,9,2),DATE(2033,5,9),2,
0)/COUPDAYS(DATE(2026,9,2),DATE(2033,5,9),2,0))/100
```

T0001's invoice: 525662.5694444444



September's trades settled for 119,283,538.57 dollars: 1,809,789.22 of accrued sits outside the principal column.

Bring it home, and what you can now do

Copy the work folder's two files into your own "my_bondmath" folder, test them there, and commit. The copy replaces your files, so anything of your own must be in the work folder first.

```
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$env:TEMP\pfi_course2_08_k4w7q2xr\bondmath.py", "$env:TEMP\pfi_course2_08_k4w7q2xr\test_bondmath.py" .
python -m pytest -q
git diff --stat
git commit -am "Add price, dirty_price, and yield_to_maturity, tested against Excel"
```

"Copy-Item" copies files, here into ".", the current folder; pytest shows 23 passed. In Colab, download the two files from the work folder under "/tmp" instead. The work folder stays in your temp folder; the system clears it in time.

You Can Now

Tell the quote from the invoice, and price the invoice in dollars.

Discount from settlement over w + k periods, in Python and in a sheet.

Price any settlement date with "price" and "dirty_price", tied to Excel's "PRICE".

Show that a new function keeps the old answers on a coupon date.

Price the final coupon period with simple interest, as Excel does.

Solve a yield with "yield_to_maturity", tied to "YIELD", and say when "YIELD" does not undo "PRICE".

Undo a committed change with "git revert".

Put the accrued back on every trade of a blotter.

Predict first: the invoice, and a par bond

Predict 1

At the same 5.40 percent yield, what does the buyer pay per 100 on 30 November? Write down a number.

```
excel_clean = week2.loc['99001BZA7', 'price']
dirty = excel_clean + quorvane_accrued
print(f'Quote (clean):   {excel_clean:.6f} per 100')
print(f'Invoice (dirty): {dirty:.6f} per 100')
print(f'Principal: {1_000_000 * excel_clean / 100:>14,.2f} dollars')
print(f'Accrued:   {1_000_000 * quorvane_accrued / 100:>14,.2f} dollars')
print(f'Invoice:   {1_000_000 * dirty / 100:>14,.2f} dollars')
```

Predict 2

Dunmarrow's coupon equals its yield, 6.125 percent, so notebook 02 priced it at 100. Its clean price on 30 November, between coupons, is:

A) exactly 100, the coupon still equals the yield

B) a little under 100

C) a little over 100

```
importlib.reload(bondmath)
maturities, coupons, yields = bonds['maturity'], bonds['coupon_pct'], bonds['yield_pct']
prices = bonds[['issuer']].copy()
prices['clean'] = [bondmath.price(settlement, m, c, y)
                   for m, c, y in zip(maturities, coupons, yields)]
prices['excel'] = week2.loc[bonds['cusip'], 'price'].to_numpy()
prices['dirty'] = [bondmath.dirty_price(settlement, m, c, y)
                   for m, c, y in zip(maturities, coupons, yields)]
assert ((prices['clean'] - prices['excel']).abs() < 1e-6).all()
prices.round(6)
```

Predict first: a colleague's edit, and the same quote

Predict 3

No running bond is in its final period. Of the 22 tests:

A) none fail

B) one fails, the final-period test

C) three fail: it and the two price tests that read every row

```
import subprocess
import sys
command = [sys.executable, '-m', 'pytest', '-q', '-p', 'no:cacheprovider', '--tb=no']
result = subprocess.run(command, cwd=work, capture_output=True, text=True)
print(result.stdout + result.stderr)
```

Predict 4

Notebook 04 found Quorvane's quote of 99.125 yields 5.452297 percent on 30 September. The quote holds until 30 November. Its yield:

A) stays the same

B) moves closer to the 5.25 coupon

C) moves further from it

```
quotes = [99.125, 100.5, 100.875, 97.75]
in_september = [bondmath.yield_to_maturity(date(2026, 9, 30), m, c, q)
                for m, c, q in zip(maturities, coupons, quotes)]
in_november = [bondmath.yield_to_maturity(settlement, m, c, q)
               for m, c, q in zip(maturities, coupons, quotes)]
change_bp = [(after - before) * 100 for before, after in zip(in_september, in_november)]
pd.DataFrame({'issuer': bonds['issuer'], 'quote': quotes, '30 Sep': in_september,
              '30 Nov': in_november, 'change_bp': change_bp}).round(6)
```

The note, answered

The note asked	Answer
A price for any date	price and dirty_price, all 306 rows of PRICE
A yield from any quote	yield_to_maturity, all 306 rows of YIELD
September's cash	119,283,538.57 dollars, 1,809,789.22 of it accrued

Be proud: you priced a bond on any day of its life, tied it to Excel on every reference row, and undid a committed mistake without losing its record.

Next: Duration and DV01 (notebook 09) asks how far that price moves for a basis point.